.

SDN STATIC ROUTING PROJECT REPORT

Title:
Static Routing using SDN Controller in Mininet

Course: Computer Networks

Name: *Adarsh Prakash*
SRN: PES2UG24CS027
Sec: *A*

---

1. Problem Statement

The objective of this project is to implement **static routing using a Software Defined Networking (SDN) controller**. The project demonstrates:

- Controller–switch interaction
- Flow rule (match–action) design
- Controlled network behavior using OpenFlow
- Static path enforcement

Unlike traditional networks, routing decisions are made centrally by the controller instead of distributed routers.

---

2. Objective

- To design a custom network topology using Mininet
- To implement an SDN controller using Ryu
- To manually define routing paths using flow rules
- To analyze network behavior under normal and failure conditions

---

3. Tools & Technologies Used

- Mininet (Network Emulator)
- Ryu Controller (Python-based SDN controller)
- Open vSwitch
- Ubuntu/Linux Environment

---

4. Network Topology

The network consists of:

- 3 Hosts: h1, h2, h3
- 4 Switches: s1, s2, s3, s4

Description:

- Multiple paths exist between nodes
- Controller enforces a **fixed static path**
- No dynamic routing is allowed

Insert Image: `net.png`

(Caption: Network topology showing hosts and switches)

---

5. Implementation Details

Controller Logic

- Controller handles `packet_in` events
- Installs flow rules using **match-action mechanism**
- Routing path is defined manually

Static Path Enforced:

```
h1 → s1 → s3 → s4 → h3
```

- Alternative paths exist but are not used
- Demonstrates centralized control in SDN

---

6. Execution Steps

Step 1: Start Controller

```
ryu-manager controller.py
```

```
seed@VM: ~/.../CDN-Mininet
[04/20/26]seed@VM:~/.../CDN-Mininet$ export PATH=$PATH:/home/seed/.local/bin
[04/20/26]seed@VM:~/.../CDN-Mininet$ ryu-manager controller.py
loading app controller.py
loading app ryu.controller.ofp_handler
instantiating app controller.py of StaticRouting
instantiating app ryu.controller.ofp_handler of OFPHandler
Switch 3, in_port 2
Flow: s3 2 -> 1
Switch 1, in_port 3
Flow: s1 3 -> 1
Switch 1, in_port 1
Flow: s1 1 -> 3
Switch 4, in_port 1
Flow: s4 1 -> 2
Switch 3, in_port 1
Flow: s3 1 -> 2
Switch 1, in_port 2
Switch 2, in_port 2
Switch 4, in_port 2
Flow: s4 2 -> 1
Switch 1, in_port 2
Switch 2, in_port 2
Switch 2, in_port 1
Switch 2, in_port 1
Switch 2, in_port 1
Switch 2, in_port 1
```

Step 2: Start Mininet

```
sudo mn --custom topo.py --topo demo --controller=remote
```

```
[04/20/26]seed@VM:~/.../CDN-Mininet$ sudo mn --custom topo.py --topo demo --controller=remote --switch ovsk,protocols=OpenFlow13
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1 s2 s3 s4
*** Adding links:
(h1, s1) (s1, s2) (s1, s3) (s2, h2) (s3, s4) (s4, h3)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 4 switches
s1 s2 s3 s4 ...
*** Starting CLI:
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.730 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.114 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.138 ms
^C
--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2018ms
rtt min/avg/max/mdev = 0.114/0.327/0.730/0.284 ms
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
^C
--- 10.0.0.2 ping statistics ---
5 packets transmitted, 0 received, +3 errors, 100% packet loss, time 4090ms
pipe 4
```

Step 3: View Network

`net`

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth2
h3 h3-eth0:s4-eth2
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth1 s1-eth3:s3-eth1
s2 lo:  s2-eth1:s1-eth2 s2-eth2:h2-eth0
s3 lo:  s3-eth1:s1-eth3 s3-eth2:s4-eth1
s4 lo:  s4-eth1:s3-eth2 s4-eth2:h3-eth0
c0
mininet>
```

7. Results & Observations

7.1 Successful Routing (h1 → h3)

`h1 ping h3`

- Packets successfully delivered
- Flow rules installed dynamically

```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.681 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.117 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.106 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=0.085 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=0.111 ms
64 bytes from 10.0.0.3: icmp_seq=7 ttl=64 time=0.115 ms
64 bytes from 10.0.0.3: icmp_seq=8 ttl=64 time=0.118 ms
64 bytes from 10.0.0.3: icmp_seq=9 ttl=64 time=0.109 ms
64 bytes from 10.0.0.3: icmp_seq=10 ttl=64 time=0.096 ms
64 bytes from 10.0.0.3: icmp_seq=11 ttl=64 time=0.132 ms
^C
--- 10.0.0.3 ping statistics ---
11 packets transmitted, 11 received, 0% packet loss, time 10244ms
rtt min/avg/max/mdev = 0.061/0.157/0.681/0.166 ms
```

.2 Unused Path Demonstration (h1 → h2)

`h1 ping h2`

- Communication not established properly
- Shows unused path

```
mininet> h1 ping h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
^C
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 0 received, +3 errors, 100% packet loss, time 5
130ms
pipe 4
mininet>
```

7.3 Flow Table Verification

`ovs-ofctl dump-flows`

- Flow rules show match-action entries
- Confirms controller behavior

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
 cookie=0x0, duration=129.928s, table=0, n_packets=64, n_bytes=8544, p
riority=10,in_port="s1-eth3" actions=output:"s1-eth1"
 cookie=0x0, duration=129.820s, table=0, n_packets=14, n_bytes=1076, p
riority=10,in_port="s1-eth1" actions=output:"s1-eth3"
 cookie=0x0, duration=130.276s, table=0, n_packets=28, n_bytes=3988, p
riority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s3
 cookie=0x0, duration=159.379s, table=0, n_packets=39, n_bytes=4802, p
riority=10,in_port="s3-eth2" actions=output:"s3-eth1"
 cookie=0x0, duration=159.267s, table=0, n_packets=40, n_bytes=4888, p
riority=10,in_port="s3-eth1" actions=output:"s3-eth2"
 cookie=0x0, duration=159.725s, table=0, n_packets=2, n_bytes=176, pri
ority=0 actions=CONTROLLER:65535
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s4
 cookie=0x0, duration=177.561s, table=0, n_packets=14, n_bytes=1076, p
riority=10,in_port="s4-eth2" actions=output:"s4-eth1"
 cookie=0x0, duration=177.449s, table=0, n_packets=66, n_bytes=8696, p
riority=10,in_port="s4-eth1" actions=output:"s4-eth2"
 cookie=0x0, duration=177.904s, table=0, n_packets=2, n_bytes=180, pri
ority=0 actions=CONTROLLER:65535
mininet>
```

7.4 Failure Scenario

```
link s3 s4 down
h1 ping h3
```

- Communication fails
- No alternate routing available

```
mininet> link s3 s4 down
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=9 Destination Host Unreachable
From 10.0.0.1 icmp_seq=10 Destination Host Unreachable
From 10.0.0.1 icmp_seq=11 Destination Host Unreachable
^C
--- 10.0.0.3 ping statistics ---
14 packets transmitted, 0 received, +3 errors, 100% packet loss, time
13306ms
pipe 4
mininet>
```

---

8. Analysis

- Static routing ensures predictable paths
- Controller provides centralized control
- No automatic rerouting in case of failure
- Flow tables confirm correct implementation

---

9. Conclusion

This project successfully demonstrates:

- SDN-based static routing
- Controller-driven flow rule installation
- Fixed path enforcement
- Network behavior under failure

The implementation highlights both advantages and limitations of static routing in SDN.

THE END